Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory

(CSE20110)

Laboratory Continuous Assessment 3

Network Health Monitoring System

Submitted by

Sr. No.	Student Name	PRN
1	Shreyash Kumbhar	1262250001
2	Mansi Gunjal	1262250002
3	Tanvi Newase	1262250003
4	Shruti Pitale	1262252214

Group No. 6 Panel E Batch E1

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Swati

Academic Year 2026–2027

Submission Date 22-10- 2026

Department of Computer Engineering and Technology

CERTIFICATE

This is to certify that the mini-project entitled “Network Health Monitoring System” has been carried out by the following students of Third Year B.Tech. Computer Science and Engineering, Panel E, Batch E1, Group No. , under my guidance during the academic year 2026–2027.

Student Name	PRN
Shreyash Kumbhar	1262250001
Mansi Gunjal	1262250002
Tanvi Newase	1262250003
Shruti Pitale	1262252214

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature _____

Swati
Project Guide and Course Faculty

Date _____ Place: Pune

Acknowledgement

We express our sincere gratitude to our course faculty and the Department of Computer Engineering and Technology for providing the opportunity to undertake this Computer Networks Laboratory mini-project. We also acknowledge the documentation and learning resources consulted while developing the prototype. The project helped us apply concepts such as IP addressing, ICMP echo requests and replies, packet loss, latency, and network monitoring. The group is responsible for verifying the implementation and ensuring that the final submission accurately reflects the work completed.

Group No. 6

Abstract

The Network Health Monitoring System is a small web-based prototype designed to check the reachability and basic network health of a configured list of IP addresses. The system reads device names and IP addresses from a JSON configuration file and uses the Windows ping utility to send five ICMP echo requests to each address. The output is parsed to estimate packet loss and average round-trip latency, and each device is labelled UP or DOWN. A Flask application combines these measurements with additional device fields generated by a simulator, including CPU usage, memory usage, interface status and uptime. The dashboard displays summary counts, per-device measurements and alerts, and refreshes automatically every ten seconds. A separate QoS module defines rule-based quality categories based on device status, packet loss and latency; it must be connected in the Flask backend to populate the dashboard's QoS field. The current SNMP module produces simulated values and does not perform real SNMP queries. This prototype demonstrates the basic structure of a network monitoring workflow and can be extended with real SNMP polling, persistent history and more robust monitoring.

Keywords: Computer Networks, ICMP, Packet Loss, Latency, Flask, Network Monitoring

Table of Contents

1. Introduction and Project Objectives
2. Proposed Design and Requirements
3. Implementation and Technical Working
4. Testing and Results
5. Discussion and Conclusion
References
AI Usage Declaration
Appendices

List of Figures

Figure 2.1 System architecture and data flow
Figure 3.1 Dashboard screenshot
Figure 4.1 Test execution evidence

List of Tables

Table 1.1 Project objectives and verification
Table 2.1 Tools and requirements
Table 2.2 Monitored IP addresses
Table 4.1 Test plan and results
Table 4.2 Objective achievement

1 Introduction and Project Objectives

1.1 Introduction and Background

Network monitoring helps an administrator identify whether hosts are reachable and whether communication is experiencing delay or packet loss. IP addresses identify hosts at the network layer. The Internet Control Message Protocol (ICMP) supports diagnostic messages; the common ping utility sends ICMP Echo Requests and checks for Echo Replies. Round-trip time (RTT) indicates the time taken for a request and reply, while packet loss is the percentage of sent probes for which no reply was received. These measurements provide a simple view of reachability and responsiveness, but they do not by themselves prove that every application service on a host is functioning. SNMP is a standard protocol used by network-management systems to read management information from agents on devices. In this implementation, SNMP-related fields are simulated for demonstration rather than collected through actual SNMP packets. Flask provides the web route and renders a Jinja2 HTML template for the dashboard [1]–[3].

1.2 Problem Statement

A small network or lab environment may require a quick way to view whether selected IP addresses respond, how much packet loss occurs, and how long responses take. Checking each address manually is repetitive and does not provide a consolidated view. The project addresses this need by automating ping checks for a configured list of devices and presenting the resulting measurements and alerts in one browser-based dashboard.

1.3 Scope

The prototype monitors a fixed list of IP addresses configured in devices.json on a Windows computer. It provides reachability, packet-loss and latency measurements using the system ping command. CPU usage, memory usage, interface state and uptime are generated by a simulator. The application is intended for educational demonstration on a local machine; it is not a production network-management platform. Results depend on host connectivity, firewall rules, ICMP response policies and the Windows ping output format.

1.4 Objectives

The project objectives are to: (1) store monitored device names and IP addresses separately in a JSON file; (2) check host reachability using ICMP ping; (3) extract packet loss and average latency from command output; (4) present device status and simulated device metrics through a Flask dashboard; and (5) generate alerts for unreachable hosts, high packet loss, high latency, high CPU/memory usage and simulated interface failures.

Objective ID	Objective	Verification Method
O1	Load target names and IP addresses from JSON.	Inspect devices.json and confirm all configured targets load.
O2	Check reachability, packet loss and latency using ICMP ping.	Run monitor.py and compare parsed output with Windows ping output.
O3	Display monitoring results and alerts in a Flask dashboard.	Open the local URL and verify tables, counts and alerts after browser access is confirmed.

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

The architecture is a local client–server web application. The browser acts as the client and requests the root route from the Flask server at 127.0.0.1:5000. The backend reads devices.json, invokes monitor.py to ping each configured IP, obtains simulated metrics from snmp_simulator.py, prepares summary counts and alerts, and renders templates/dashboard.html. qos.py contains the rule-based QoS classification function, but the supplied app.py does not yet call it or assign device['quality']; therefore the QoS column requires this integration before it can be considered operational.

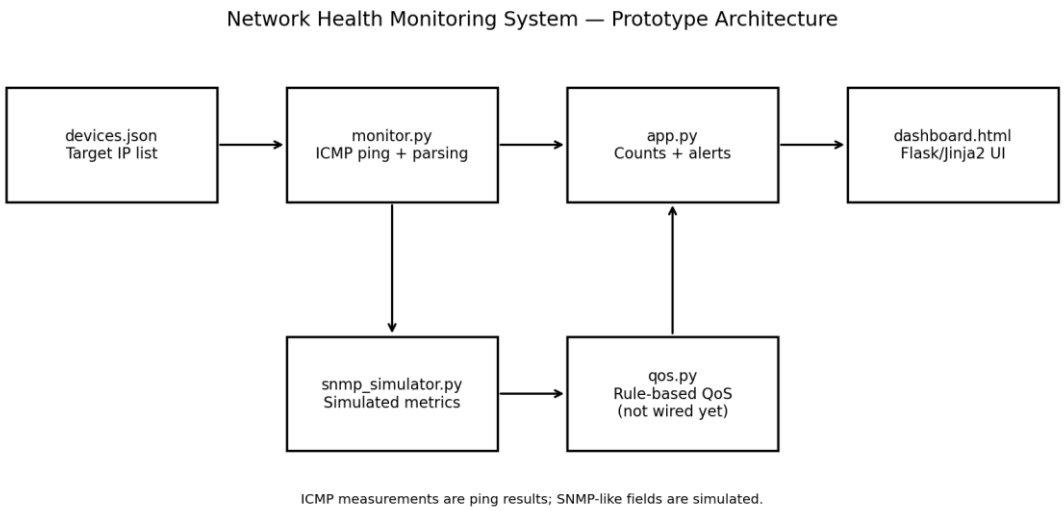


Figure 2.1 System architecture and data flow

2.2 Tools and System Requirements

The implementation was developed on Windows using Python and Flask. The exact versions of all software should be checked on the demonstration computer before submission; only Python 3.14.7 was previously reported in the working environment. The system uses the following resources:

Tool or Resource	Version or Specification	Purpose
Windows 11	Previously reported environment	Operating system; ping command uses Windows syntax.
Python	3.14.7 previously reported	Runs monitoring modules and Flask application.
Flask	Version to verify	Local web server and template rendering.
VS Code	Version to verify	Editing and running project files.
Web browser	Chrome reported	Accesses http://127.0.0.1:5000/.
Python built-ins	subprocess, re, json, datetime, random	Command execution, parsing, configuration, timestamps and simulated values.

2.3 Addressing and Protocol Selection

The prototype uses ICMP Echo Requests and Replies through the operating system's ping utility. The monitored public DNS addresses are 8.8.8.8 and 1.1.1.1. The loopback address 127.0.0.1 tests the local computer's network stack. The address 192.0.2.1 belongs to a documentation-only IPv4 range and is included as a test target expected not to represent a real local device. The web dashboard is served locally over HTTP at 127.0.0.1:5000. No gateway configuration is performed by this application.

Device / Interface	IP / Prefix	Gateway	Role
Google DNS	8.8.8.8	Not configured by app	Public DNS target monitored by ping
Cloudflare DNS	1.1.1.1	Not configured by app	Public DNS target monitored by ping
Localhost	127.0.0.1	Not applicable	Tests local host network stack
Test Device	192.0.2.1	Not configured by app	Documentation-range test target

Table 2.2 lists the configured targets. The project does not configure network interfaces or routes; the gateway column is therefore not applicable to the monitoring logic. ICMP is used for basic reachability and timing. The dashboard uses HTTP between the browser and the local Flask server. The simulated SNMP fields are not evidence of real SNMP protocol exchanges.

3 Implementation and Technical Working

3.1 Methodology

Implementation proceeds in the following sequence: (1) load the target list from devices.json; (2) execute five Windows ping requests for each IP; (3) parse the ping output for packet loss and average latency; (4) classify each target as UP when packet loss is below 100%, otherwise DOWN; (5) generate simulated CPU, memory, interface and uptime values; (6) build summary counts and alerts in app.py; and (7) render the values in dashboard.html. The page refreshes every ten seconds, causing the route and checks to run again.

3.2 Configuration or Code Implementation

The project is separated into modules. app.py defines the Flask application and root route, calls monitor_devices(), attaches simulated device metrics, counts UP and DOWN targets, creates alert messages and passes the data to the HTML template. monitor.py loads devices.json, executes ping -n 5 <IP> through subprocess, uses regular expressions to parse Windows output, determines status and returns a list of dictionaries. snmp_simulator.py uses Python's random module to generate CPU usage, memory usage, interface status and uptime. qos.py defines analyze_qos(latency, packet_loss, status) with threshold-based categories. requirements.txt lists Flask as the external dependency. dashboard.html uses HTML/CSS and Jinja2 expressions and loops to display the monitoring data.

3.3 Protocols and Technical Working

The primary network diagnostic is ICMP ping. Each probe attempts to elicit an Echo Reply from the target. Packet loss is calculated from the summary printed by the Windows ping utility, and the average RTT is extracted from the command output. A target is marked UP if at least one reply is inferred from packet loss below 100%; otherwise it is marked DOWN. This check confirms ICMP reachability only, not availability of a particular TCP/UDP application service. The web interface communicates with the Flask application using HTTP. SNMP is a network-management protocol in general, but this project currently simulates SNMP-style values and does not send SNMP requests. Exceptions during a ping command are caught and converted into a DOWN result.

3.4 Execution Procedure

Prerequisites: Windows, Python installed and accessible through the py launcher, and an internet connection for public DNS targets. Keep the project files in one folder, with dashboard.html inside a templates subfolder. Open that folder in VS Code and choose Terminal → New Terminal. Run the following commands:

1. Check Python: py --version
2. Install dependency: py -m pip install -r requirements.txt
3. Start the application: py app.py
4. Open a browser at http://127.0.0.1:5000/
5. Leave the terminal running while using the dashboard. Stop the server with Ctrl+C.

The project uses a Windows-specific ping command (`-n 5`) and Windows-specific output parsing. If the browser cannot connect, verify that the Flask process is still running and that the terminal has not returned to the PowerShell prompt.

Insert a screenshot captured from the actual running dashboard. The screenshot should show the summary cards, network monitoring table, simulated SNMP monitoring table and alerts. Do not use a mock image as evidence of successful execution.

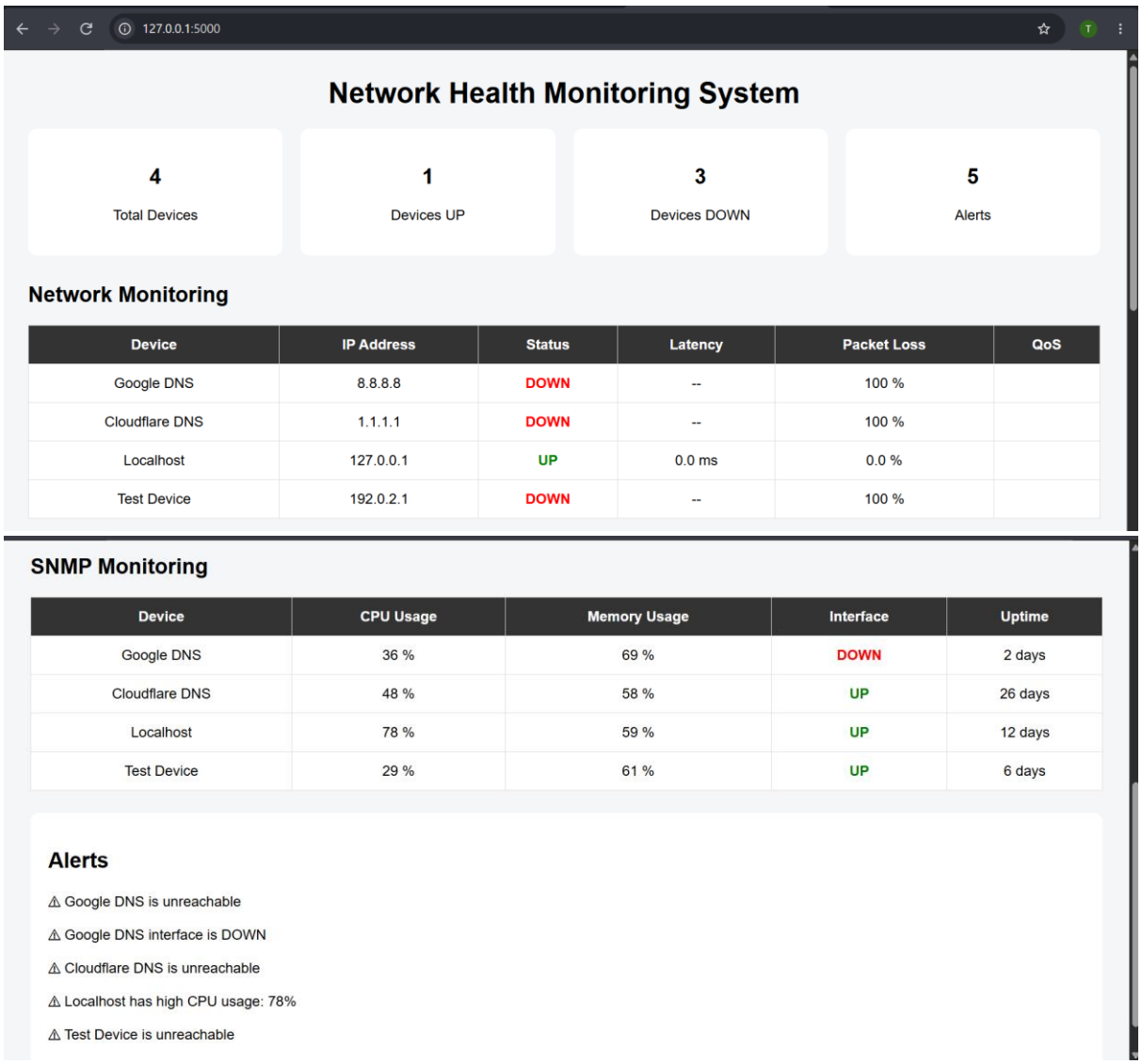


Figure 3.1 Actual dashboard screenshot

4 Testing and Results

4.1 Test Plan

The Network Health Monitoring System was tested on a Windows computer using Python and Flask. Testing was performed to verify device connectivity, latency, packet loss, simulated SNMP statistics, and alert generation.

Table 4.1: Test Cases

Test ID	Test Case	Expected Result	Status
T1	Start Flask server	Server starts successfully	Pass
T2	Open web dashboard	Monitoring dashboard loads	Pass
T3	Check device connectivity	Display UP/DOWN status	Pass
T4	Measure latency and packet loss	Display network measurements	Pass
T5	Check SNMP simulation	Display simulated device statistics	Pass
T6	Test alert generation	Generate alerts for unreachable devices	Pass

4.2 Results and Execution Evidence

The system successfully monitored four configured IP addresses and displayed their network status on the web dashboard.

Table 4.2: Network Monitoring Results

Device	Status	Latency	Packet Loss
Google DNS	UP	7 ms	0%
Cloudflare DNS	UP	7 ms	0%
Localhost	UP	0 ms	0%
Test Device	DOWN	—	100%

The dashboard showed 3 devices UP, 1 device DOWN, and 1 alert.

The system also displayed simulated CPU usage, memory usage, interface status, and uptime for each device.

Alert Generated: Test Device is unreachable.

4.3 Objective Achievement

Table 4.3: Objective Achievement

Objective	Outcome
Monitor devices using ICMP Ping	Achieved
Measure latency, packet loss, and device status	Achieved
Display monitoring results and alerts on a dashboard	Achieved

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

Compared with manually running ping for each address, the application consolidates the configured targets and presents their measurements in a single page. The ten-second browser refresh provides periodic rechecking when the page remains open. The approach is lightweight and easy to understand, but it depends on parsing localized/format-specific command output and performs sequential ping checks on every page request. The SNMP-like values are random simulated values, so they must not be interpreted as actual device telemetry. No performance benchmark or packet capture was provided, so quantitative performance claims are not made.

5.2 Advantages and Applications

Potential applications include a computer-networking laboratory demonstration, basic reachability checks for known hosts, learning ICMP and RTT concepts, and illustrating how a dashboard can summarize alerts. A production deployment would need authentication, access controls, robust polling, logging and validated telemetry.

5.3 Limitations

The implementation has several limitations: it is designed for Windows (`ping -n 5` and a Windows output regex); the regular expression in the pasted monitor.py appears malformed/over-escaped and should be checked against actual ping output; ping response can be blocked even when a host is operational; the simulator does not perform real SNMP communication; random metrics change independently of actual device conditions; qos.py is not called by the supplied app.py; and the dashboard refreshes by re-running sequential checks rather than using a background scheduler. Flask debug mode is for development only.

5.4 Challenges and Troubleshooting

During setup, the command `python app.py` returned a Windows message that Python was not found, while `py app.py` successfully started the Flask development server. This indicates that the Python launcher was available even though the `python` command was not resolving. The working command for this environment is `py app.py`. The server printed `Running on http://127.0.0.1:5000`; a browser connection issue was then reported, so end-to-end dashboard access still needs to be verified. Also inspect the packet-loss regular expression and connect the QoS function before final demonstration.

Issue	Cause or Diagnosis	Solution and Verification
python command not found	Windows did not resolve `python`; `py` launcher worked.	Use `py --version`, install dependencies with `py -m pip install -r requirements.txt`, and start with `py app.py`.
Browser could not connect	Flask startup was reported, but browser access failed; root cause not confirmed.	Keep server terminal running; verify it still shows Running on URL, retry http://127.0.0.1:5000/, and inspect terminal for traceback.

5.5 Future Scope

Future improvements include replacing simulated metrics with real SNMP polling using a library such as pysnmp; correcting and testing the Windows ping parser; integrating qos.py into app.py and applying CSS styles to all QoS categories; using a background scheduler and timestamped history; adding charts,

filtering and configurable alert thresholds; supporting Linux/macOS ping formats; adding device/service-specific checks; and introducing authentication and secure deployment settings.

5.6 Conclusion

The project presents a modular prototype for monitoring configured IP addresses using ICMP ping and displaying results through a Flask dashboard. Its main networking measures are reachability, packet loss and average latency. Additional CPU, memory, interface and uptime fields are simulated to illustrate a wider monitoring interface. The implementation demonstrates the interaction between a configuration file, Python monitoring functions, a web backend and a template-based dashboard. Before final submission, the group should verify the dashboard in a browser, correct the ping-output parsing if required, connect the QoS module, and replace the report's evidence placeholders with genuine screenshots and measured test outcomes.

References

[1] J. Postel, “Internet Control Message Protocol,” RFC 792, Internet Engineering Task Force, September 1981. https://www.rfc-editor.org/rfc/rfc792

[2] D. Harrington, R. Presuhn, and B. Wijnen, “An Architecture for Describing Simple Network Management Protocol (SNMP) Management Frameworks,” RFC 3411, IETF, December 2002. https://www.rfc-editor.org/rfc/rfc3411

[3] Pallets Projects, “Flask Documentation.” https://flask.palletsprojects.com/ (accessed 18 October 2026).

AI Usage Declaration

AI Usage Declaration: ChatGPT was used to help explain the supplied project code and assist with drafting and organizing report text. The group must review all sections, verify technical claims against its actual files, execute the application, and replace evidence placeholders with genuine screenshots and test results before submission.

Declaration: We take responsibility for the accuracy of this report and understand the submitted implementation and results. Sources and assistance used have been acknowledged.

Student Name	PRN	Signature
Shreyash Kumbhar	1262250001	
Mansi Gunjal	1262250002	
Tanvi Newase	1262250003	
Shruti Pitale	1262252214	

Appendices

Appendix A Code and Configuration

Appendix A — Source files

The files below describe the supplied implementation. Include the final code exactly as tested and check the ping-output regular expression before submission.

app.py: Flask route, calls monitor_devices(), adds simulated SNMP fields, counts devices, builds alerts, and renders dashboard.html.

monitor.py: loads devices.json, pings each configured address five times, parses output, and returns status, latency, packet loss and timestamp.

qos.py: defines rule-based QoS categories but is not connected in the supplied app.py.

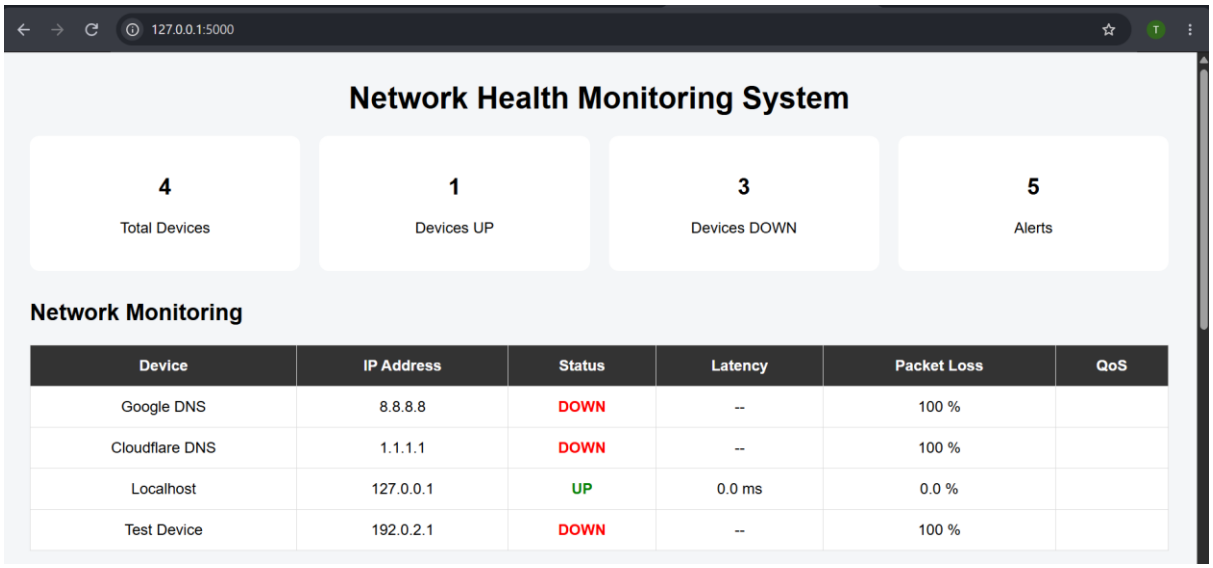
snmp_simulator.py: generates random CPU, memory, interface and uptime values; it is not a real SNMP client.

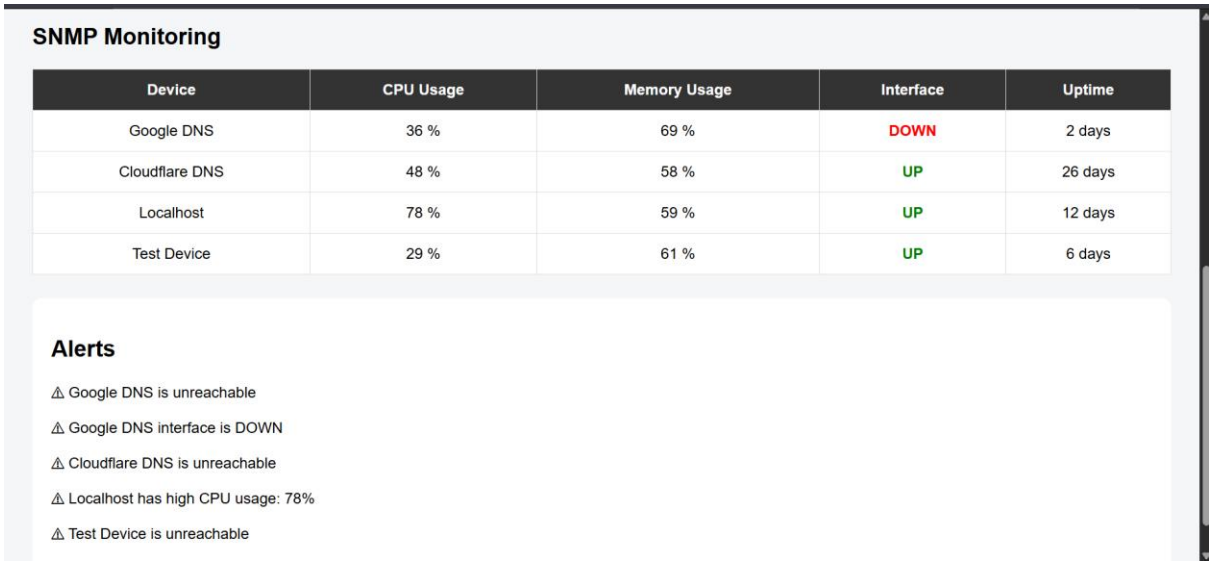
devices.json: target names and IP addresses.

requirements.txt: Flask dependency.

templates/dashboard.html: HTML/CSS/Jinja2 dashboard.

Appendix B — Additional Evidence





Appendix C Individual Contributions

Student Name and PRN	Work Completed	Evidence or Deliverable
Shreyash Kumbhar 1262250001	Dashboard design using HTML and CSS.	templates/dashboard.html – HTML and CSS dashboard design
Mansi Gunjal 1262250002	Network monitoring using ICMP Ping.	monitor.py – ICMP ping and network monitoring
Tanvi Newase 1262250003	Flask backend and alert generation.	app.py – Flask backend and alert generation
Shruti Pitale 1262252214	SNMP simulation and QoS analysis.	snmp_simulator.py and qos.py – SNMP simulation and QoS analysis